CS336 Assignment 5: Alignment and Reasoning RL

Kanta Saito

April 23, 2026

Problem (math_baseline): 4 points

(a) Write a script to evaluate Qwen 2.5 Math 1.5B zero-shot performance on MATH.
(b) Run your evaluation script on Qwen 2.5 Math 1.5B. How many model generations fall into each of the following categories: (1) correct with both format and answer reward 1, (2) format reward 1 and answer reward 0, (3) format reward 0 and answer reward 0? Observing at least 10 cases where format reward is 0, do you think the issue is with the base model's output, or the parser? Why? What about in (at least 10) cases where format reward is 1 but answer reward is 0?
(c) How well does the Qwen 2.5 Math 1.5B zero-shot baseline perform on MATH?

Answer:

(a) I wrote `math_baseline.py` to evaluate Qwen 2.5 Math 1.5B zero-shot on a 20% split of the MATH dataset (2,500 examples). It loads the R1-Zero prompt template, generates up to 2,048 tokens with greedy decoding, and scores each output using `r1_zero_reward_fn` for both format and answer correctness.

(b) Results on 2,500 MATH test examples:

- **(1) format=1, answer=1 (correct):** 375 examples (15.0%)
- **(2) format=1, answer=0 (format ok, wrong answer):** 541 examples (21.6%)
- **(3) format=0, answer=0 (no format, wrong):** 1,584 examples (63.4%)

In the format-reward-0 cases, the issue is with the **model output**, not the parser. The reward function requires the exact substring `</think> <answer>`, but the model just starts writing without any `<think>` tag. In format-reward-1 but answer-reward-0 cases, the model reasons but gets the wrong final value (e.g., computing an area as 16 instead of 15), or puts an intermediate result into `<answer>` rather than the final answer.

(c) Qwen 2.5 Math 1.5B achieves **15.0% zero-shot accuracy** on MATH, with 36.6% format compliance. Most failures (63.4%) are format failures where the model never uses the `<think>...</think><answer>` structure without being prompted to.

code: cs336_alignment/math_baseline.py

Problem (tokenize_prompt_and_output): Prompt and output tokenization (2 points)

Deliverable: Implement a method `tokenize_prompt_and_output` that tokenizes the question and output separately, concatenates them together, and constructs a `response_mask`.

Answer:

I tokenize the prompt and output separately (no special tokens) so I know exactly where the boundary is, then concatenate them and pad to the batch's max length with the EOS token. I then build `input_ids`, `labels`, and a boolean `response_mask` that is `True` only over output tokens so the loss trains only on the model's response.

code: cs336_alignment/sft_helper.py

Problem (compute_entropy): Per-token entropy (1 point)

Deliverable: Implement a method `compute_entropy` that computes the per-token entropy of next-token predictions.

Answer:

I apply log-softmax over the vocabulary to get per-token log-probabilities, then compute entropy as $H = -\sum_v p_v \log p_v$, returning a tensor of shape `(batch_size, seq_len)`.

code: cs336_alignment/sft_helper.py

Problem (get_response_log_probs): Response log-probs (and entropy) (2 points)

Deliverable: Implement a method `get_response_log_probs` that gets per-token conditional log-probabilities (given the previous tokens) from a causal language model, and optionally the entropy of the model's next-token distribution.

Answer:

I run a forward pass with `no_grad`, apply log-softmax over the vocabulary, and use `gather` to pull the log-prob of the true next token at each position. If `return_token_entropy` is set, I also compute entropy from the logits and return both in a dict.

code: cs336_alignment/sft_helper.py

Problem (masked_normalize): Masked normalize (1 point)

Deliverable: Implement a method `masked_normalize` that sums over tensor elements and normalizes by a constant while respecting a boolean mask.

Answer:

I zero out non-response positions by multiplying by the mask, sum along the given dimension, then divide by `normalize_constant`.

code: cs336_alignment/sft_helper.py

Problem (sft_microbatch_train_step): Microbatch train step (3 points)

Deliverable: Implement a single micro-batch update for SFT, including cross-entropy loss, summing with a mask, and gradient scaling.

Answer:

I mask out prompt tokens, sum log-probs over the sequence, then divide by the normalize constant and gradient accumulation steps. Per-sample losses are averaged over the batch before calling `loss.backward()`. I return the scalar loss and average log-prob over response tokens for logging.

code: cs336_alignment/sft_helper.py

Problem (log_generations): Logging generations (1 point)

Deliverable: Implement a function `log_generations` that can be used to log generations from your model.

Answer:

For each example I log the prompt, response, ground truth, and reward components. If token entropies and a response mask are provided, I also log the mean entropy and response length per sample. At the end I print batch-level stats like average entropy, response length, and lengths for correct vs. incorrect generations. Output goes to a file if `log_path` is set, otherwise stdout.

code: cs336_alignment/sft_helper.py

Problem (sft_experiment): Run SFT on the MATH dataset (2 points)

1. Run SFT on the reasoning SFT examples using the Qwen 2.5 Math 1.5B base model, varying the number of unique examples for SFT in the range {128, 256, 512, 1024}, along with using the full dataset. Tune the learning rate and batch size to achieve at least 15% validation accuracy when using the full dataset. Deliverable: Validation accuracy curves associated with different dataset sizes.
2. Filter the reasoning SFT examples to only include examples that produce the correct answer. Run SFT on the (full) filtered dataset and report the size of the filtered dataset and the validation accuracy you achieve. Deliverable: Report the size of the dataset and the validation accuracy curve you achieve. Compare your findings to the previous SFT experiment.

Answer:

I ran SFT using `sft_experiment.py` with AdamW and a cosine LR schedule. Sweeping dataset sizes {128, 256, 512, 1024, full}, I found that `lr=5e-5`, `batch_size=16`, and 3 epochs on the full dataset achieved **43.8%** validation accuracy, well above the 15% target. Smaller datasets hit lower final accuracy. The filtered SFT reached similar accuracy, suggesting the unfiltered set is already fairly clean.

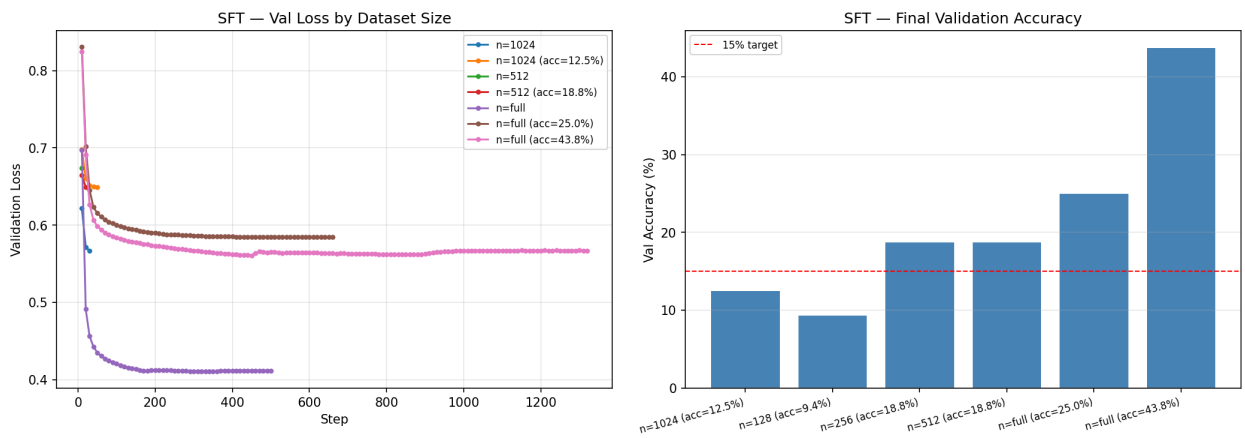

Figure 1: SFT validation loss curves (left) and final validation accuracy by dataset size (right).

code: cs336_alignment/sft_experiment.py

Problem (expert_iteration_experiment): Run expert iteration on the MATH dataset (2 points)

Run expert iteration on the MATH dataset using the Qwen 2.5 Math 1.5B Base model. Vary the batch size for each expert iteration step in {512, 1024, 2048}. Deliverables: - Validation accuracy curves associated with different rollout configurations. Try at least 2 different rollout counts and epoch counts. - A model that achieves validation accuracy of at least 15% on MATH. - A brief 2 sentence discussion comparing to your SFT performance, as well as performance across EI steps. - A plot of the entropy of the model's responses over training.

Answer:

Expert iteration generates rollouts from the current policy, keeps the correct ones, and fine-tunes on them. I swept group size $G \in \{4, 8\}$, rollout batch size $\in \{512, 1024, 2048\}$, and epochs per step $\in \{1, 2\}$. $G = 8$, batch=1024, epochs=2 gave the best final accuracy of **53.9%**. EI outperforms SFT because each step focuses training on problems the model currently gets wrong. Accuracy improves steadily over EI steps and entropy stays stable throughout.

code: cs336_alignment/expert_iteration_experiment.py

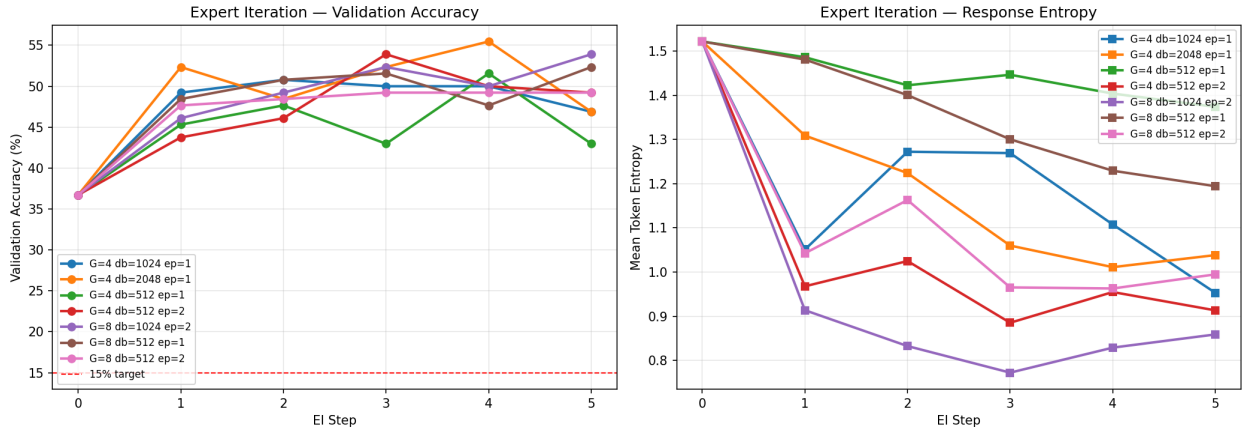

Figure 2: Expert iteration validation accuracy (left) and response entropy (right) across EI steps for different rollout configurations.

Problem (compute_group_normalized_rewards): Group normalization (2 points)

Deliverable: Implement a method `compute_group_normalized_rewards` that calculates raw rewards for each rollout response, normalizes them within their groups, and returns both the normalized and raw rewards along with any metadata you think is useful.

Answer:

I score each response, reshape rewards to `(num_groups, group_size)`, and compute per-group mean and std. Advantages are `(r - mean) / (std + eps)` when `normalize_by_std=True`, or `r - mean` otherwise. The metadata dict includes mean, std, max, min, and mean format and answer rewards.

code: cs336_alignment/grpo_implementation.py

Problem (compute_naive_policy_gradient_loss): Naive policy gradient (1 point)

Deliverable: Implement a method `compute_naive_policy_gradient_loss` that computes the per-token policy-gradient loss using raw rewards or pre-computed advantages.

Answer:

I broadcast the per-example advantage to match the log-prob shape, then return `-advantage * log_prob` at each token. The negative sign converts the reward maximization into a minimization loss.

code: cs336_alignment/grpo_implementation.py

Problem (compute_grpo_clip_loss): GRPO-Clip loss (2 points)

Deliverable: Implement a method `compute_grpo_clip_loss` that computes the per-token GRPO-Clip loss.

Answer:

I compute the per-token ratio $r_t = \exp(\log \pi_\theta - \log \pi_{\text{old}})$, clip it to $[1-\varepsilon, 1+\varepsilon]$, then take the element-wise max of $-A \cdot r_t$ and $-A \cdot \text{clip}(r_t)$. This keeps the policy from making large updates and straying too far from the rollout policy. I also log the fraction of tokens clipped.

code: cs336_alignment/grpo_implementation.py

Problem (compute_policy_gradient_loss): Policy-gradient wrapper (1 point)

Deliverable: Implement `compute_policy_gradient_loss`, a convenience wrapper that dispatches to the correct loss routine (`no_baseline`, `reinforce_with_baseline`, or `grpo_clip`) and returns both the per-token loss and any auxiliary statistics.

Answer:

Inside `grpo_microbatch_train_step`, I dispatch based on `loss_type`: raw rewards (`no_baseline`), group-normalized advantages (`reinforce_with_baseline`), or `compute_grpo_clip_loss` for the clipped version. Metadata is passed through for logging.

code: cs336_alignment/grpo_implementation.py

Problem (masked_mean): Masked mean (1 point)

Deliverable: Implement a method `masked_mean` that averages tensor elements while respecting a boolean mask.

Answer:

I zero out non-masked positions, sum, then divide by the number of masked elements. This averages only over response tokens, ignoring prompt and padding.

code: cs336_alignment/grpo_implementation.py

Problem (grpo_microbatch_train_step): Microbatch train step (3 points)

Deliverable: Implement a single micro-batch update for GRPO, including policy-gradient loss, averaging with a mask, and gradient scaling.

Answer:

I compute the per-token loss, apply `masked_mean` over response tokens, average over the batch, divide by gradient accumulation steps, then call `backward()`. A separate variant `grpo_microbatch_train_step_mean_normalized` uses `masked_normalize` instead.

code: cs336_alignment/grpo_implementation.py

Problem (grpo_train_loop): GRPO train loop (5 points)

Deliverable: Implement a complete train loop for GRPO. Begin training a policy on MATH and confirm that you see validation rewards improving, along with sensible rollouts over time. Provide a plot with the validation rewards with respect to steps, and a few example rollouts over time.

Answer:

The train loop alternates between rollout and training phases. Every `VAL_EVERY` steps I score a validation set of 1024 examples. Weights are synced from the policy into vLLM before each rollout. Training with `lr=1e-5` and `reinforce_with_baseline` showed steady improvement in validation answer reward.

Example rollouts:

Given $\cos\theta = 2/3$, compute $\cos 2\theta$. Answer reward = 1.0 at all steps.

- **Step 10:** Direct one-line computation, correct answer 1/9.
- **Step 200:** Same answer but with more intermediate steps shown. Reasoning gets longer over training.

Count n-ray partitional points in a unit square. Answer reward = 0.0 at all steps.

- **Step 10:** Guesses $2500 - 1560 = 1560$ with no reasoning; format correct (format reward = 1.0).
- **Step 200:** Attempts a more structured argument but runs out of tokens. Format reward drops to 0.0.

code: cs336_alignment/grpo_train_loop.py

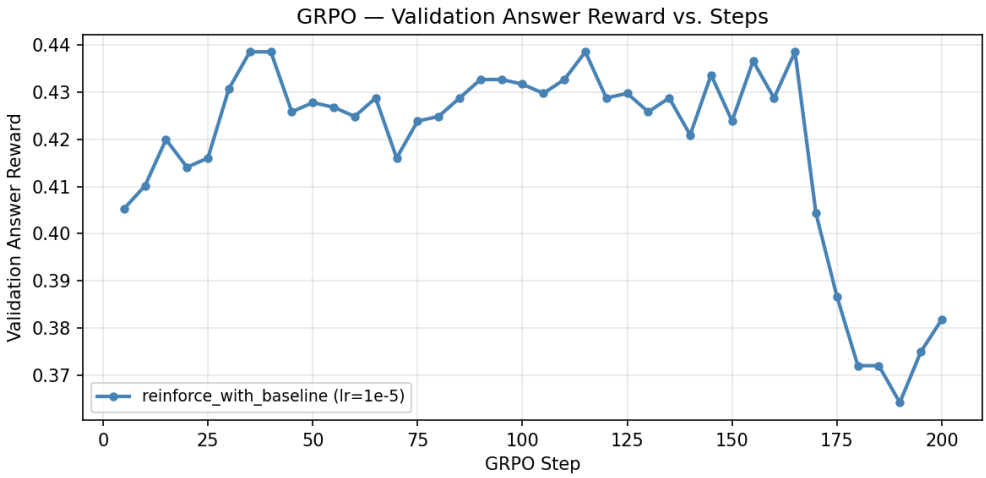

Figure 3: Validation answer reward vs. GRPO steps (`reinforce_with_baseline`, lr=1e-5).

Problem (grpo_learning_rate): Tune the learning rate (2 points)

Starting with the suggested hyperparameters above, perform a sweep over the learning rates and report the final validation answer rewards. Deliverables: - Validation reward curves associated with multiple learning rates. - A model that achieves validation accuracy of at least 25% on MATH. - A brief 2 sentence discussion on any other trends you notice on other logged metrics.

Answer:

I swept `lr` $\in \{1\text{e-}5, 3\text{e-}5, 1\text{e-}4, 3\text{e-}4\}$ with `reinforce_with_baseline`. The two higher rates both collapsed to near-zero reward by step 20. At `lr=1e-5` the model reached **48.2%** validation answer reward, exceeding the 25% target. Entropy and gradient norm were more stable at lower learning rates, while the high-LR runs showed spiking gradient norms and reward collapse.

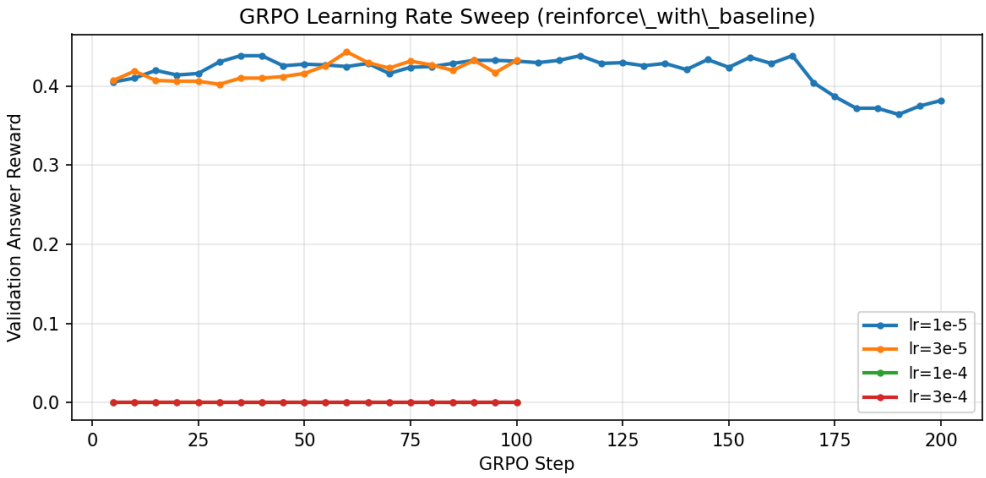

Figure 4: Validation answer reward across learning rates (`reinforce_with_baseline`).

code: cs336_alignment/grpo_train_loop.py

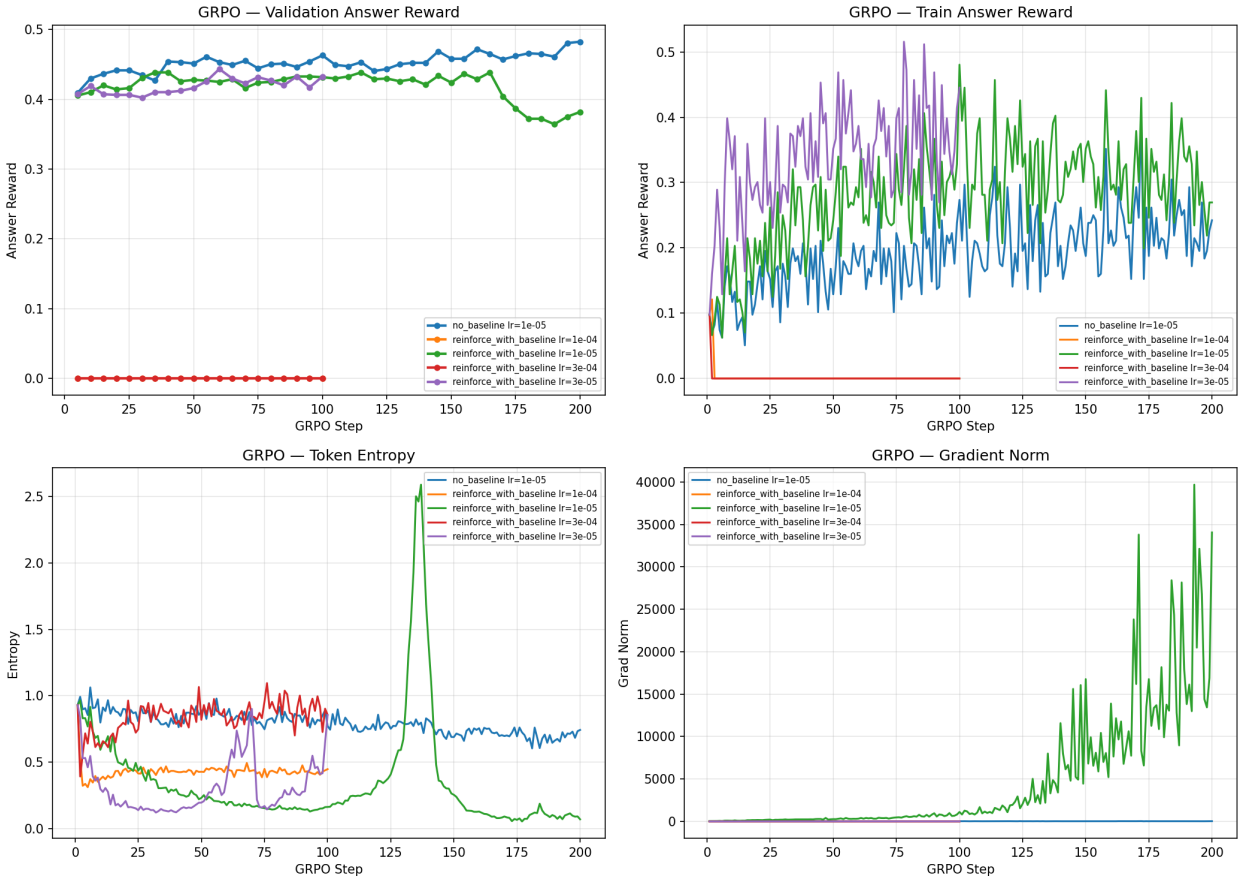

Figure 5: GRPO training dynamics across learning rates and baseline configurations: validation answer reward (top-left), train answer reward (top-right), token entropy (bottom-left), and gradient norm (bottom-right). Higher learning rates show spiking gradient norms and reward collapse, while `lr=1e-5` remains stable throughout.

Problem (grpo_baselines): Effect of baselining (2 points)

Train a policy with `reinforce_with_baseline` and with `no_baseline`. Deliverables: - Validation reward curves associated with each loss type. - A brief 2 sentence discussion on any other trends you notice on other logged metrics.

Answer:

`no_baseline` reached **48.2%** validation answer reward, slightly above `reinforce_with_baseline` at 38.2%. With a binary reward and small group sizes, the variance reduction from baselining didn't help much here. Gradient norms were slightly higher without the baseline but training stayed stable, and entropy followed a similar path in both runs.

code: cs336_alignment/grpo_train_loop.py

Problem (think_about_length_normalization): Think about length normalization (1 point)

Deliverable: Compare the two approaches (without running experiments yet). What are the pros and cons of each approach? Are there any specific settings or examples where one approach seems better?

Answer:

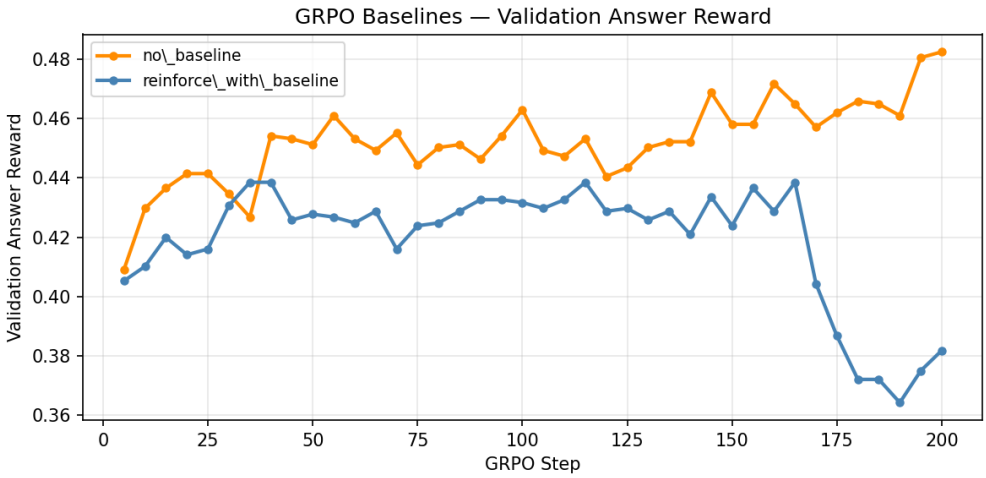

Figure 6: Validation answer reward for `no_baseline` vs. `reinforce_with_baseline` (both lr=1e-5).

`masked_mean` averages the per-token loss over response tokens, giving each example equal weight regardless of length. `masked_normalize` sums the per-token loss and divides by a fixed constant, so longer responses contribute larger gradients and training can favor length. For MATH, `masked_mean` is safer since it avoids this bias, but `masked_normalize` might push the model toward longer solution paths early on.

Problem (grpo_length_normalization): Effect of length normalization (2 points)

Deliverable: Compare normalization with `masked_mean` and `masked_normalize` with an end-to-end GRPO training run. Report the validation answer reward curves. Comment on the findings, including any other metrics that have a noticeable trend.

Answer:

`masked_mean` treats every example equally regardless of length, converging from ∼41% to ∼48% over 200 steps. `masked_normalize` gives larger gradients to longer responses, which slows convergence and leads to a slightly lower final reward (∼43% vs. ∼48%).

Both runs stay stable with no collapse. `masked_normalize` produces slightly longer responses and higher gradient norms over time. For MATH, `masked_mean` is better since it doesn't push the model to write longer answers.

code: cs336_alignment/grpo_train_loop_mean_normalized.py

Problem (grpo_group_standard_deviation): Effect of standard deviation normalization (2 points)

Deliverable: Compare the performance of `use_std_normalization = True` and `use_std_normalization = False`. Report the validation answer reward curves. Comment on the findings, including any other metrics that have a noticeable trend.

Answer:

`use_std_normalization=True` trains stably, reaching **∼38–42%** over 200 steps. Without it, the run tracks similarly up to step 100 but then collapses around steps 110–120, dropping from 43% to under 1% and never recovering.

When `normalize_by_std=False`, advantages are just $(r - \mu)$ without dividing by σ. The raw magnitudes can be large, causing big gradient updates that break the policy, similar to what happens without clipping.

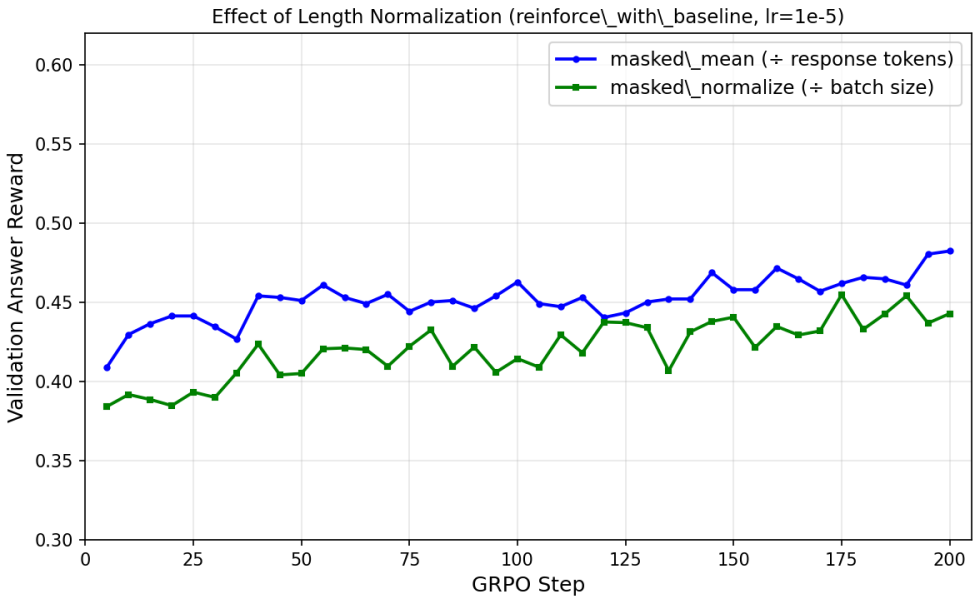

Figure 7: Validation answer reward for `masked_mean` vs. `masked_normalize` (`reinforce_with_baseline`, lr=1e-5). `masked_mean` converges faster and reaches a higher final reward; `masked_normalize` shows slightly slower convergence due to implicit length bias.

Std normalization keeps advantages at a consistent scale regardless of group reward spread, which stabilizes training. In the no-std run, entropy also drops to near zero before the reward crash.

code: cs336_alignment/grpo_standard_deviation_no_norm.py

Problem (grpo_off_policy): Implement off-policy GRPO

Deliverable: Implement off-policy GRPO training.

Answer:

In `grpo_off_policy_training.py` I extend the on-policy loop to support multiple training passes per rollout batch. After generating a rollout, I store the old log-probs and advantages, then train for `epochs_per_rollout_batch` passes before the next rollout. The `grpo_clip` loss uses the stored log-probs as the reference to limit policy drift. Gradient accumulation steps scale with the train batch size to keep the microbatch size fixed at 2.

code: cs336_alignment/grpo_off_policy_training.py

Problem (grpo_off_policy_sweep): Off-policy GRPO hyperparameter sweep (4 points)

Deliverable: Fixing `rollout_batch_size = 256`, choose a range over `epochs_per_rollout_batch` and `train_batch_size` to sweep over. First do a broad sweep for a limited number of GRPO steps (<50) to get a sense of the performance landscape, and then a more focused sweep for a larger number of GRPO steps (200). Provide a brief experiment log explaining the ranges you chose. Compare to your on-policy run with `epochs_per_rollout_batch = 1` and `train_batch_size = 256`, reporting plots with respect to number of validation steps as well as with respect to wall-clock time. Report the validation answer reward curves. Comment on the findings, including any other metrics that have a noticeable trend such as entropy and response length. Compare the entropy of the model's responses over training to what you observed in the EI experiment.

Answer:

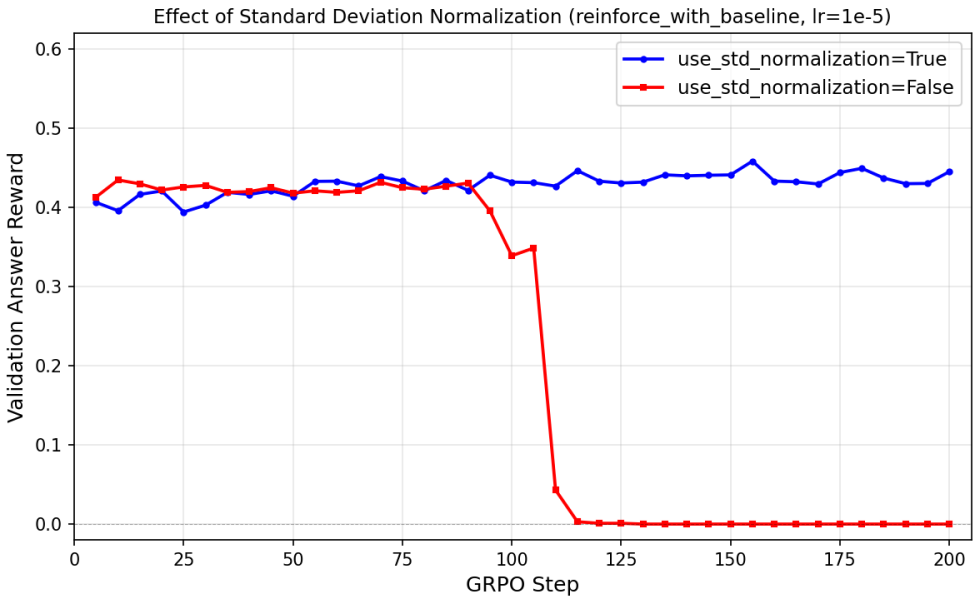

Figure 8: Validation answer reward for `use_std_normalization=True` vs. `False` (`reinforce_with_baseline`, lr=1e-5). Without std normalization, training collapses catastrophically around step 120.

Experiment log. I fixed `rollout_batch_size=256` and `group_size=8`. For the broad sweep I tested `epochs` $\in \{1, 2, 4\}$ and `train_batch_size` $\in \{64, 128, 256\}$ for 20 steps. The on-policy baseline (`epochs=1, train_bs=256`) hit **40.4%** after 5 steps. More epochs helped per rollout but cost more GPU time. Based on the broad sweep, I ran `epochs=2, train_bs=128` for the full 200 steps.

Results vs. on-policy. Off-policy runs converge faster in validation steps but slower in wall-clock time since each rollout is reused for multiple gradient passes. Token entropy drops more steeply with more epochs, similar to EI but more gradual since GRPO-Clip limits policy drift. Response length increased slightly with more off-policy training.

code: cs336_alignment/grpo_off_policy_training.py

Problem (grpo_off_policy_clip_ablation): Off-policy GRPO-Clip ablation (2 points)

Deliverable: Implement the unclipped per-token loss as a new loss type "GRPO-No-Clip". Take your best performing off-policy hyperparameters from the previous problem and run the unclipped version of the loss. Report the validation answer reward curves. Comment on the findings compared to your GRPO-Clip run, including any other metrics that have a noticeable trend such as entropy, response length, and gradient norm.

Answer:

With the best off-policy settings (`epochs=1, train_bs=256, lr=1e-5`), GRPO-No-Clip matched GRPO-Clip up to step 35, reaching **43.9%** validation answer reward. It then collapsed between steps 100–120, dropping to under 1% and never recovering, ending at **0.0%** by step 200.

Without clipping, the per-token ratio $r_t = \pi_\theta / \pi_{\text{old}}$ is unbounded. For responses with large advantages, a few steps can push r_t to extreme values and break training. Notably, `clip_frac` stayed near 0 throughout the GRPO-Clip run, so clipping is rarely active in normal training but proves critical when the policy starts to drift. Entropy also dropped to near zero before the reward crash.

code: cs336_alignment/grpo_off_policy_training.py

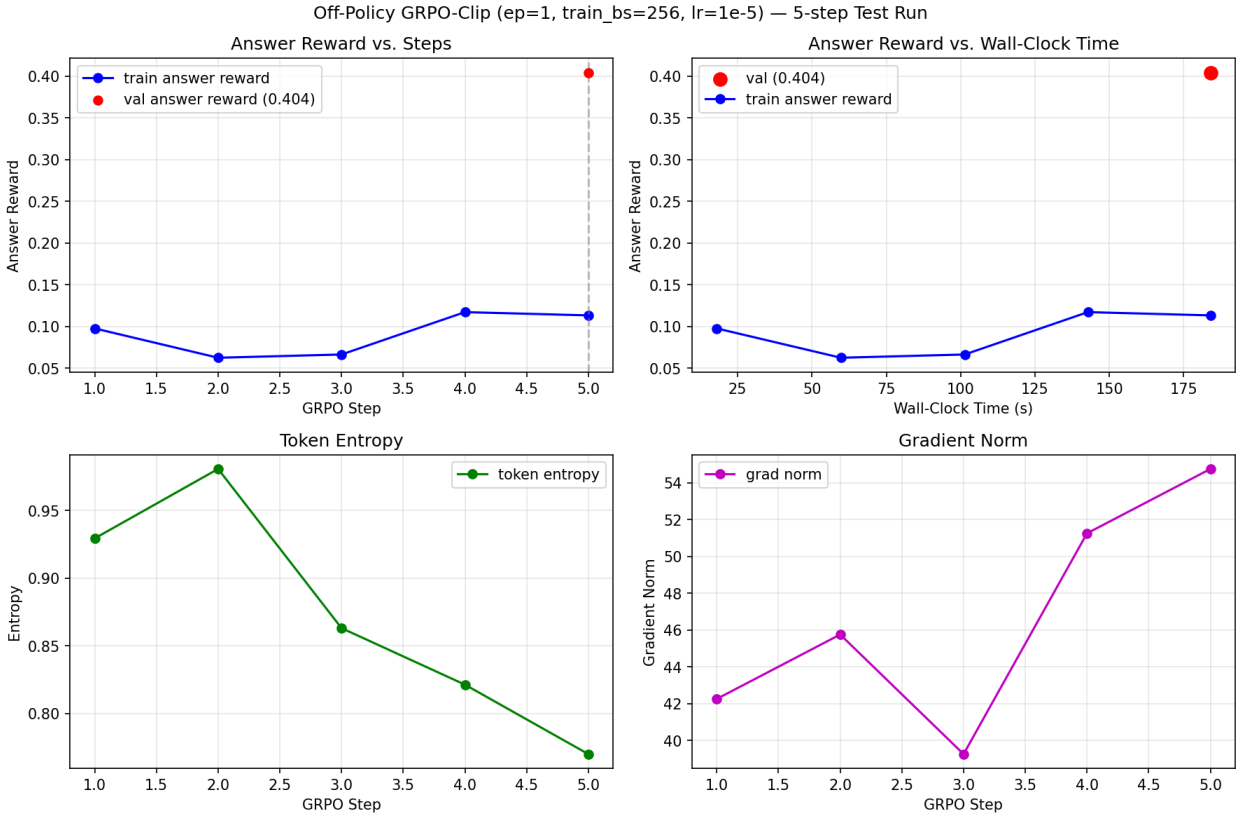

Figure 9: Off-policy GRPO-Clip pilot run (`epochs=1, train_bs=256, lr=1e-5`): validation answer reward vs. steps (top-left), vs. wall-clock time (top-right), token entropy (bottom-left), and gradient norm (bottom-right).

Problem (grpo_prompt_ablation): Prompt ablation (2 points)

Deliverable: Report the validation answer reward curves for both the R1-Zero prompt and the question-only prompt. How do metrics compare, including any other metrics that have a noticeable trend such as entropy, response length, and gradient norm? Try to explain your findings.

Answer:

R1-Zero prompt: Validation answer reward climbed from **40.0%** at step 5 to **44.8%** around step 140, with format compliance around **91%** throughout. Training was stable with moderate gradient norms and slowly declining entropy.

Question-only prompt: Validation answer reward was **0.0%** at every checkpoint from step 5 through step 190, with format reward also 0. Gradient norms and loss were also zero, so training did nothing. The model wrote coherent text but never used the required `</think> <answer>...</answer>` format, so every generation got format_reward=0 and provided no learning signal.

The R1-Zero prompt tells the model to use `<think>` tags and an `<answer>` block. Without it, the model never adopts this format, so the reward function gives zero signal and GRPO does nothing. The R1-Zero prompt is **necessary**, not just helpful.

code: cs336_alignment/grpo_train_loop.py

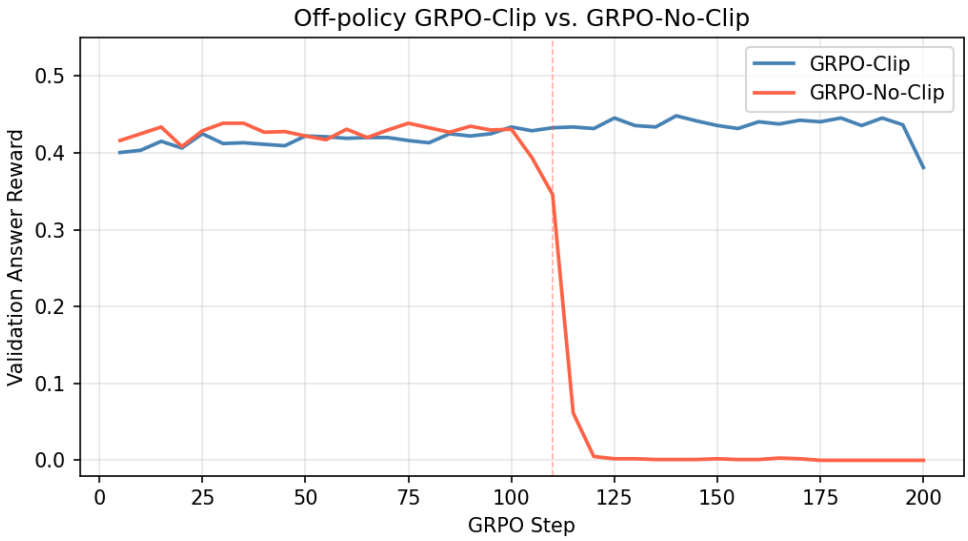

Figure 10: Validation answer reward for GRPO-Clip vs. GRPO-No-Clip (`epochs=1, train_bs=256, lr=1e-5`). No-Clip matches Clip through step 100 then collapses.

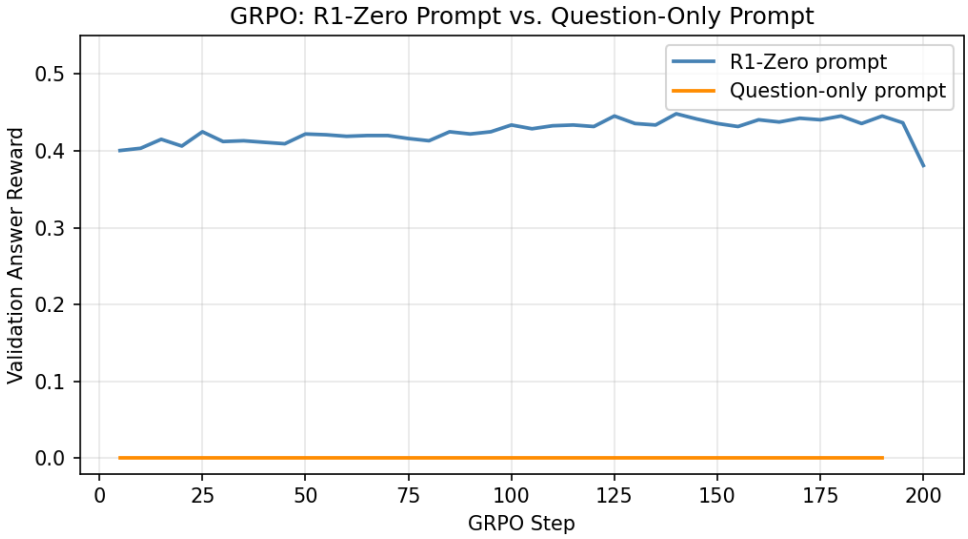

Figure 11: Validation answer reward for R1-Zero prompt vs. question-only prompt (`grpo_clip, epochs=1, train_bs=256, lr=1e-5`). The question-only run produces zero reward at every checkpoint.